

Assignment No	22
Title	Make use of hook useEffect
Objective	A. To change the background and side effect useEffect to change color of circle B. To perform calculations as side effect when counter is incremented
Roll No	MCA2511

A. To change the background and side effect useEffect to change the color of the circle.

Code:

[ColorChanger.js](#)

```
import React, { useState, useEffect } from "react";

const ColorChange = () => {
  const [circleBackColor, setCircleBackColor] = useState("red");

  const getRandomColor = () => {
    return "#" + Math.random().toString(16).slice(2, 8);
  };

  const changeColor = () => {
    setCircleBackColor(getRandomColor());
  };

  useEffect(() => {
    document.body.style.backgroundColor = getRandomColor();
  });

  return (
    <>
      <div>ColorChange</div>
      <div
        style={{height:"150px", cursor:"pointer", width: "150px",
borderRadius: "100%", backgroundColor: circleBackColor}}
        className="circle"
      >
      </div>
    </>
  );
};
```

```
        onClick={changeColor}
      ></div>
      <div className="txt">👉 Click Me</div>
    </>
  );
};

export default ColorChange;
```

index.js

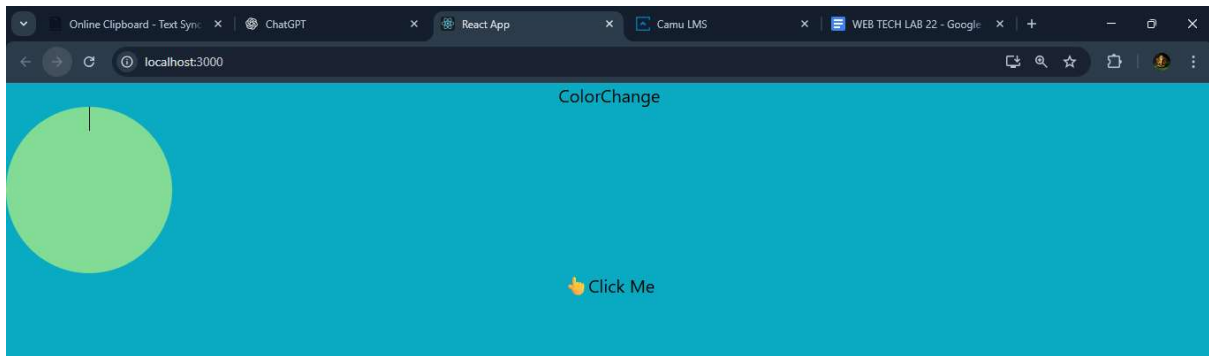
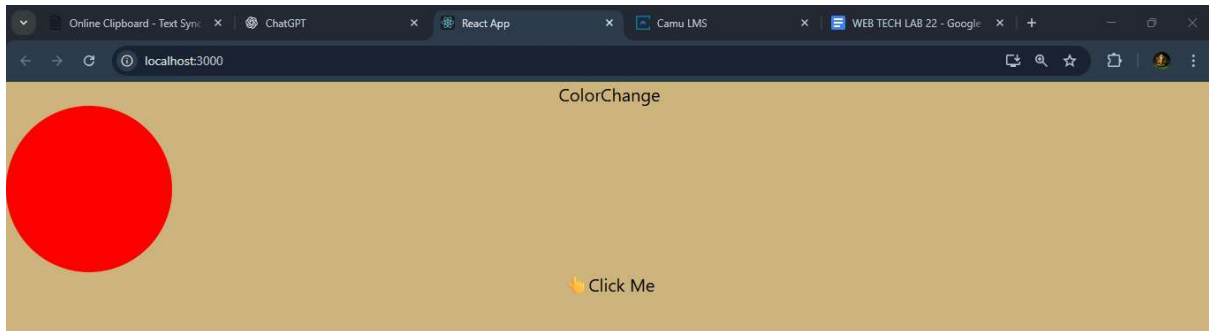
```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
import reportWebVitals from './reportWebVitals';

import './App.css';
import ColorChanger from './Components/ColorChanger';
import CounterEffect from './Components/ColorEffect';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <ColorChanger/>\
  </React.StrictMode>
);

// If you want to start measuring performance in your app, pass a
function
// to log results (for example: reportWebVitals(console.log))
// or send to an analytics endpoint. Learn more:
https://bit.ly/CRA-vitals
reportWebVitals();
```

Output:



B. To perform calculations as a side effect when the counter is incremented.

Code:

ColorEffect.js

```
import { useState, useEffect } from 'react';
function CounterEffect() {
  const [count, setCount] = useState(0);
  const [calculation, setCalculation] = useState(0);
  useEffect (() => {
    setCalculation(() => count + 2*3);

  },[count]); //<- add the count variable here
  return (
    <>
      <p>Count: {count}</p>
      <button onClick={() => setCount((c) => c+1)}>+</button>
      <p>Calculation: {calculation}</p>
    </>
  )
}
export default CounterEffect;
```

index.js

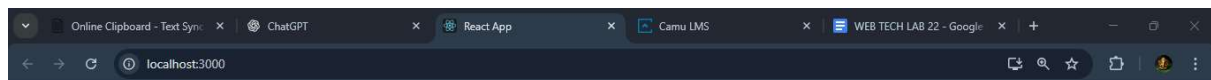
```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
import reportWebVitals from './reportWebVitals';

import './App.css';
import ColorChanger from './Components/ColorChanger';
import CounterEffect from './Components/ColorEffect';

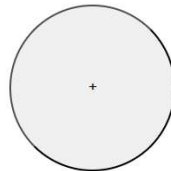
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <CounterEffect/>
  </React.StrictMode>
);
```

```
// If you want to start measuring performance in your app, pass a
function
// to log results (for example: reportWebVitals(console.log))
// or send to an analytics endpoint. Learn more:
https://bit.ly/CRA-vitals
reportWebVitals();
```

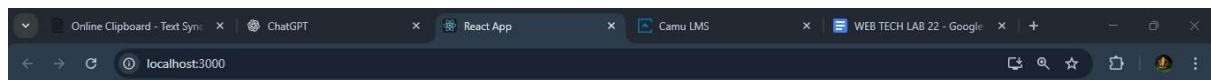
Output:



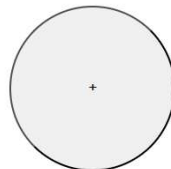
Count: 0



Calculation: 6



Count: 1



Calculation: 7